

Mini-Project 3 - Understanding Data and Model Steering

Submit answers for this mini-project as a PDF document that has at most 3 pages in the main part (and a possible appendix that doesn't count towards the 3-page limit) and code (including a README file that explains the code structure) via the following Google Form: <https://forms.gle/LxHmthGHWLXPeeQU9>. Please make only one submission per team and clearly mention the names and university email addresses of the team members. **The submission deadline is June 10 (Wednesday) at 23:59.** Mini-project presentations will take place during the exercise session on June 15.

Part 1: Understanding and Filtering Pretraining Data (5 points)

Your task. Investigate and annotate pretraining datasets and ‘actually look at the data’. Check 50 random samples from each of the four sources listed in Resources below, annotate them along several dimensions, and then run OpenAI’s privacy filter over the same samples to compare automated detection against your own judgment.

Background. Modern LLMs are trained on large web-scraped corpora whose contents are impossible to fully inspect. Datasets like the Common Crawl derivatives (C4, FineWeb, RedPajama) are produced via heuristic and model-based cleaning pipelines from raw HTML, and the quality, topical distribution, and presence of private or harmful material in the surviving samples shape downstream model behavior. Looking at the data by hand is one way to discover failure modes and to get a feeling for what’s going into the model. Complementing manual inspection, model-based filters such as OpenAI’s recently released **privacy-filter** aim to scrub PII automatically.

Sampling setup. Use **streaming** or **sample** mode when accessing each dataset, so you don’t have to download terabytes. Draw at least 30 random documents per source (so 120 in total), recording their dataset IDs from FineWeb, FinePDFS, C4 and Redpajama (linked below), and read them.

1.1 Data Analysis (3 points)

- **[2 points] a)** For each of the >120 samples, produce a row in a CSV file recording: a) dataset source, b) document index, c) topic, d) quality (1–10), whether the sample was correctly cleaned from its original HTML form (e.g., is crucial context missing or is anything malformed), e) whether the sample contains information you consider (A) private or (B) harmful, e) number of characters, and f) number of sentences.
- **[1 point] b)** Write-up: Afterwards, write up your findings. What patterns do you observe (if any). What surprised you after reading these samples.

1.2 OpenAI Privacy Filter (2 points)

- **[1 point] a)** OpenAI recently released a transformer model that can be used to annotate and censor private information. Download OpenAI’s privacy-filter model

(huggingface.co/openai/privacy-filter) and run it on the same samples from the previous task. Compare its predictions against your own privacy annotations from 1.1. Catalogue cases where (i) the filter flags content you consider a **false positive**, and (ii) the filter skips content you marked as private (a **false negative**).

- [1 point] b) Summarize your findings in a write-up.

Resources

- FineWeb: huggingface.co/datasets/HuggingFaceFW/fineweb
- FinePDFs: huggingface.co/datasets/HuggingFaceFW/finepdfs
- C4 (colossal, cleaned Common Crawl): huggingface.co/datasets/allenai/c4
- RedPajama-V2 sample: huggingface.co/datasets/togethercomputer/RedPajama-Data-V2/tree/main/sample
- OpenAI privacy-filter: huggingface.co/openai/privacy-filter

Part 2: Representation Engineering and Activation Steering (5 points)

Your task. Investigate how high-level concepts (e.g., refusal, harmfulness, sentiment, truthfulness) are encoded in LLM activations and how those representations can be manipulated to steer model behavior. Use Qwen3-4B as the base model (or any small open-weight chat model).

Background. A growing body of work (Zou et al. 2023; Panickssery et al. 2023; Ardit et al. 2024) shows that LLMs encode many high-level concepts as linear directions in their residual stream. Two complementary techniques exploit this: **probing**, training a linear classifier on hidden states to test whether and where a concept is linearly encoded and to detect this concept from later inputs; and **activation steering**, adding or subtracting a vector from the residual stream during the forward pass to push the model toward (or away from) a behavior. The vector can be obtained as a *difference of means* between activations on positive and negative examples; Ardit et al. showed that a single such direction suffices to control refusal. You will need forward hooks on residual-stream modules (e.g., `model.model.layers[i]`); plain PyTorch hooks work.

There are two common ways to use a steering vector v at inference time. **Additive steering** (Panickssery et al. 2023) adds $\alpha \cdot v$ to the residual stream to *induce* the behavior, or subtracts $\alpha \cdot v$ (equivalently, uses a negative coefficient) to *suppress* it; here α is a scalar coefficient controlling intervention strength. **Directional ablation** (Ardit et al. 2024) instead projects the direction out of the residual stream h entirely ($h \leftarrow h - \hat{v} \hat{v}^T h$, with $\hat{v}$ the unit vector along v), across all layers and token positions, removing the component along v rather than just nudging it. For (iii) "suppressing" in 2.2a) below you may use either approach; in 2.2c) you will compare the two directly.

Dataset splits. Build your contrastive dataset of ~100–200 examples per class and split it into a **train set** (used to train the logistic-regression probes in 2.1 and to compute the steering vector in 2.2) and a **test set** (used to evaluate probe accuracy in 2.1 and as the held-out prompts for generations in 2.2). Use the same split throughout for consistency.

2.1 Probing for concepts (2 points)

- **[1 point] a)** Pick a binary concept (e.g., tone, harmfulness, refusal) and build the contrastive dataset described above (e.g., harmful instructions from AdvBench vs. harmless from Alpaca; true vs. false factual claims; you can use existing datasets or use an LLM to generate examples for a specific concept). Cache the residual-stream activation at the **last prompt token** for every layer, i.e., the final input token fed to the model (the position just before generation would start). This does **not** have to be the same character across prompts, but apply your chat template consistently so the final few tokens have the same role (e.g., the end-of-template marker). Train one logistic-regression probe per layer **on the train split**, evaluate probing accuracy **on the test split**, and plot accuracy vs. layer depth.
- **[1 point] b)** Write-up: At which layer is the concept most linearly decodable? Is there a clear best layer or a broad plateau? Repeat probing at 2–3 other token positions (first, mid-sequence); how does the picture change? Briefly discuss what this suggests about *where* and *when* the concept is computed.

2.2 Activation Steering (3 points)

- **[1 point] a)** From the same contrastive dataset (train split), compute a steering vector via difference-of-means at one or two well-chosen layers according to the previous probing experiment. Generate ~30 responses on **held-out prompts**. Use the prompts from the **test split** of your contrastive dataset (or a separate set of generation prompts if more appropriate for your concept). Compare three conditions: (i) baseline, (ii) inducing via additive steering, (iii) suppressing via either negative additive steering or directional ablation. You may follow Arditi et al. and use refusal as the target, or pick another concept. Score behavior change with a keyword check (e.g., refusal markers) or an LLM-as-judge. **If using an LLM judge, validate it by manually classifying 20–30 generations per condition.** A single calibration of the judge on the 2.2a) generations is sufficient; you may reuse the same judge in 2.2b) without re-validating.
- **[1 point] b)** **For additive steering**, run two ablations: (*layer sweep*) apply the same vector at every layer from early to late and measure where steering is most effective; (*coefficient sweep*) vary the scaling coefficient α (the scalar multiplying the steering vector before it is added to the residual stream) and plot behavior-change rate against a simple coherence score. **Bonus (optional):** additionally compare last-token-only vs. all-token interventions.
- **[1 point] c)** Write-up: At which layer and coefficient does steering work best; is there a sweet spot? **Bonus (optional): Side effects:** does steering one concept perturb unrelated capabilities? E.g., if you ablate harmfulness, does the model also become less factual, less coherent, or shift in tone? Pick 1–2 **side-effect measures** (a handful of MMLU questions, sentiment, perplexity on WikiText) and report.
- **[1 point]** Finally, compare additive steering with directional ablation: which is more selective, which more disruptive?

Fallback. If activations from Qwen3-4B are too slow, drop to a smaller chat model (Qwen2.5-0.5B-Instruct).

Resources

- Arditi et al. 2024, “Refusal in Language Models Is Mediated by a Single Direction”: arxiv.org/abs/2406.11717
- Panickssery et al. 2023, “Steering Llama 2 via Contrastive Activation Addition”: arxiv.org/abs/2312.06681

- Zou et al. 2023, “Representation Engineering”: arxiv.org/abs/2310.01405
- Qwen3-4B: huggingface.co/Qwen/Qwen3-4B
- Qwen2.5-0.5B-Instruct: huggingface.co/Qwen/Qwen2.5-0.5B-Instruct
- AdvBench: github.com/llm-attacks/llm-attacks; Alpaca: huggingface.co/datasets/tatsu-lab/alpaca